Reviewer Pack — Minimal Rank of Free Probability Spaces vs Resolution Proof …

Entry 73ad3e05af5d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 06:15:57 UTC

Minimal Rank of Free Probability Spaces vs Resolution Proof Width

Entry ID: `73ad3e05af5d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 06:15:57 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['For any CNF formula F with n variables, the minimal rank r(F) of its associated free probability space is upper-bounded by a function φ(n) such that log_2(t*(F)) ≤ r(F) ≤ φ(n), where t*(F) denotes the width of the smallest resolution proof for F.', 'The conjecture further states that there exists an absolute constant c > 0 such that for all CNFs, φ(n) ≥ c·log_2(n).', 'Lastly, the resolution proof complexity of F can be approximated by r(F) to within a multiplicative factor of log_2(n).']

Rationale (proposer's reasoning):

> ['Free probability theory offers a non-commutative generalization of classical probability theory that has been applied to quantum information and operator algebras. This conjecture suggests that free probability spaces could provide a new lens through which to understand the complexity of resolution proofs, potentially leading to novel approaches for proof complexity lower bounds.', 'If true, this would establish a quantitative relationship between free probability and resolution proof complexity, which are currently studied independently.']

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d6907dea6ccfaa36`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a CNF formula F with n variables, if log_2(t*(F)) ≤ r(F) and r(F) ≤ c·log_2(n), where t*(F) is the width of the smallest resolution proof for F, c is an absolute constant, and φ(n) = c·log_2(n) is the upper bound function, then the conjecture is supported. If any seed produces a metric violating log_2(t*(F)) ≤ r(F) or r(F) ≤ φ(n), the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "free probability theory" AND "resolution proof complexity" AND minimal rank" - "minimal rank free probability space" AND CNF resolution proof" - "approximation resolution proof complexity minimal rank free probability space"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_kcnf(n, k):
        variables = list(range(1, n + 1))
        clauses = []
        for _ in range(k):
            clause = [random.choice(variables), random.choice([-x for x in variables])]
            if len(set(clause)) == 2:
                clauses.append(clause)
        return clauses

    def matrix_rank(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = 0
        for i in range(min(m, n)):
            if all(matrix[j][i] == 0 for j in range(i, m)):
                continue
            pivot_row = i
            while matrix[pivot_row][i] == 0:
                pivot_row += 1
            matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
            for j in range(m):
                if j != i:
                    factor = -matrix[j][i] / matrix[i][i]
                    for k in range(n):
```

```
                    matrix[j][k] += factor * matrix[i][k]
        rank += 1
    return rank

def resolution_width(clauses):
    queue = clauses[:]
    while queue:
        clause = queue.pop(0)
        new_clauses = []
        for other_clause in queue:
            common_vars = set(x for x in clause if -x in other_clause)
            if len(common_vars) == 1:
                literal = list(common_vars)[0]
                new_clause = [x for x in clause if x != literal] + [x for x in other_clause if
                if new_clause not in queue and new_clause not in new_clauses:
                    new_clauses.append(new_clause)
        queue.extend(new_clauses)
    return len(queue)

n = random.choice([10, 20, 30, 40])
k = int(1.5 * n)  # Ensure at least 3 clauses per variable
cnf = generate_kcnf(n, k)
t_star = resolution_width(cnf)

if t_star == 0:
    return {
        "metric_name": "t_star",
        "metric_value": t_star,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "resolution_width_is_zero"
    }

matrix = [[random.choice([-1, 0, 1]) for _ in range(n)] for _ in range(n)]
r_F = matrix_rank(matrix)

if r_F == 0:
    return {
        "metric_name": "r_F",
        "metric_value": r_F,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "matrix_rank_is_zero"
    }

c = Fraction(1, 2)  # Example constant
phi_n = c * math.log2(n)

conjecture_holds = (math.log2(t_star) <= r_F) and (r_F <= phi_n)
counterexample = "" if conjecture_holds else "t_star={} r_F={}".format(t_star, r_F)

return {
    "metric_name": "r_F",
    "metric_value": r_F,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
```

```
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print("TRIAL: seed={}, {}".format(seed, result))
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_value, std_value,
    elif support_fraction >= 0.8:
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_value, std_value,
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print("RESULT: FALSIFIED counterexample=\"{}\" first_failing_seed={}".format(results[first
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it did not complete its execution to verify the conjecture. | next: Run the test again with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13179
2	preregistration	ollama_remote	glm4:latest	0	0	6806
3	novelty	ollama_remote	glm4:latest	0	0	4564
4	novelty	ollama_remote	glm4:latest	0	0	5373
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19971
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12153
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12505
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12531
9	judge	ollama_remote	glm4:latest	0	0	8367

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 95448 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/73ad3e05af5d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/73ad3e05af5d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/73ad3e05af5d.tar.gz` (if generated)